HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementierung und Evaluation der Reinforcement-Learning-Methoden Self-Play und Liga-Training in der MicroRTS-Umgebung

Name!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Mathematisch-Naturwissenschaftliche Fakultät

XXXXXXXXXXX

Abstract

This thesis improves an existing agent for the real-time strategy game (RTS) MicroRTS. RTS games are considered a challenging domain for Deep Reinforcement Learning (DRL), since agents must deal with large combinatorial action spaces, simultaneous decisions, as well as delayed and often sparse rewards. As a baseline method, we use Proximal Policy Optimization (PPO). In many DRL training setups, such as pure PPO, one reaches a point at which the agent cannot be trained further efficiently using PPO alone, because there are not enough strong opponents to train against or because the available opponents are not diverse enough to develop a robust policy. An obvious and simple countermeasure is Self-Play (SP) however, in practice it can be unstable and lead to overfitting or cyclical strategies, which can reduce performance against external opponents.

League Training (LT) stabilizes SP by building a population of agents across different training stages and roles. The training agent is then deliberately matched against diverse opponents. Among others, against specialized agents that were trained to exploit weaknesses of the current policy. We evaluate the resulting agents in matches against various opponent types and compare LT with pure SP as well as with our initial agent. Our results show that LT increases the robustness and generalization capability of the agent against different opponents.

Contents

1. Introduction

The initial agent was trained using a mixture of Behavior Cloning (BC) and PPO.

Reinforcement Learning (RL) has established itself in recent years as a powerful approach for developing agents in complex environments. Game environments such as RTS games pose a particular challenge in this regard, with large action spaces, long-term planning, and simultaneous decisions. However, they also provide clearly defined state spaces, reward structures, and repeatable experiments, which makes them an ideal and challenging test environment for researching RL methods.

MicroRTS is a simplified version of RTS games that was developed specifically for research [Huang et al., 2021]. It is discrete and reduces the visual and mechanical complexity of large commercial RTS games, such as StarCraft II. At the same time, it retains the core difficulties, for example resource management, unit production, combat mechanics, and tactical positioning. MicroRTS is therefore suitable for the systematic investigation of learning methods. MicroRTS also contains numerous training challenges, such as large combinatorial action spaces, delayed rewards, multi-agent interactions, and it also requires robust strategies against different opponents. For example, the agent must decide early on whether it wants to play aggressively or defensively against the current opponent, which affects the overall game strategy and requires long-term planning.

A central problem in RL is generalization across different opponents and strategies: agents often learn strategies that are strongly tailored to the training opponent or a specific game situation. This leads to overfitting and limited robustness. SP, Fictitious Self-Play (FSP), and LT are approaches that address this problem. In SP, the agent trains by playing against itself, which leads to a continuous learning process that does not require additional diverse or strong opponents. However, training is often unstable because of cyclical learning.

To reduce the instabilities of pure SP, the agent in FSP is not trained against the current agent but against previous versions of itself, which prevents cyclical learning [Heinrich et al., 2015]. LT extends FSP by training the agent against historical opponents from a league that consists of different agent types, for example policy snapshots from earlier training stages as well as specialized agents [Vinyals et al., 2019]. The goal is to stabilize the learning environment, in which agents compete against a broad spectrum of strategies and thereby become more robust.

Despite the theoretical advantages, the practical implementation of LT in complex environments such as MicroRTS is associated with significant challenges. These include:

- the efficient management and selection of opponents for training the Main Agent and for the specialized agents,
- stabilizing the training process under non-constant opponent distributions,
- dealing with large action spaces and sparse or delayed rewards, and

In addition, we will compare and evaluate, the learning curve, strategic diversity, and robustness for the different training approaches.

The base agent used in this project is trained in a three-stage procedure that combines BC and BC fine-tuning (BC-FT), followed by fine-tuning with PPO: The two BC phases enable the agent to quickly learn a competent base strategy by imitating expert behavior [Torabi et al., 2018]. This training approach reduces the initial, often inefficient exploration in DRL [Torabi et al., 2018]. Learning from scratch with DRL methods such as PPO can be very time-consuming [Martin and Jhala, 2021]. PPO is a well-known on-policy policy optimization method that clips the policy update size to stabilize policy updates [Schulman et al., 2017].

PPO further improves the performance of the BC agent through exploration and adaptation to the environment, allowing the agent to adapt and generalize beyond the expert strategies [Martin and Jhala, 2021]. This two-stage approach, in which a policy pre-trained with BC is refined by a DRL algorithm, has proven effective in reducing training time and improving overall performance compared to a pure DRL approach [Martin and Jhala, 2021].

2. Related Work

Game Playing Agents in MicroRTS The (unpublished) bachelor's thesis by M. Huft that underlies this work develops a competitive agent for MicroRTS and evaluates the hybrid training pipeline consisting of BC, BC-FT, and PPO for the base agent from the introduction [Huft, 2025].

The results show a clear performance increase across all three training phases (Table **??**): BC mainly learns basic game mechanics, while BC-FT improves generalization against multiple bots. Finally, PPO achieves high win rates against many of the built-in bots [Huft, 2025].

There still are agents in the results against which the base agent does not perform well. The thesis also identifies limitations, in particular the focus on a single map and the limited opponent diversity, and proposes extensions using self-play or league mechanisms. Exactly this proposal is taken up by the present work by extending the existing base agent and the initial code with SP and LT. The MicroRTS built-in bots against which evaluation is conducted include RandomBiasedAI, WorkerRushAI, LightRushAI, and CoacAI.

Table 1. Winrates (%) for BC, BC-FT, and PPO reported in [Huft, 2025].

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

MicroRTS-Py (Gym-μRTS)

Huang et al. provide an efficient Gym-compatible interface for our environment MicroRTS [Ontañón, 2013] with *Gym-µRTS / MicroRTS-Py* [Huang et al., 2021]. MicroRTS-Py is an established and widely used DRL baseline that enables the investigation of DRL approaches in full real-time strategy game scenarios without requiring extensive technical infrastructure such as high-performance computing clusters [Huang et al., 2021]. Furthermore, the *openai/baselines* [Dhariwal et al., 2017] implementation of PPO was adapted in order to fit MicroRTS-Py. We use a large part of this implementation in LT. It was also previously used in the PPO training phases of the base agent [Huft, 2025]. On top of openai/baselines, *invalid action masking* and suitable action/observation representations (e.g., GridNet [Han et al., 2019]) as well as architectural choices (such as IMPALA-CNN-a deep convolutional neural network structure with residual blocks for processing grid-based state representations in RL) [Espeholt et al., 2018] were added [Huang et al., 2021]. In a modified form, these are still used in the LT code. The paper also considered using *Unit Action Simulation* (UAS) instead of GridNet. UAS iterates over all units at each step and returns the possible actions for each unit, which substantially reduces the action space. GridNet, in contrast, predicts an action for each cell on the map. This means that GridNet executes its large action space in a single step, but this also simplefies the training [Huang et al., 2021, Han et al., 2019]. Against the built-in bots of MicroRTS, which are based on heuristic rules, the PPO implementation with invalid action masking and IMPALA-CNN achieves an average win rate of 91% with UAS and 84% with GridNet. Ultimately, the decision was made to focus on GridNet, since its complexity is lower, and it facilitates compatibility with SP (and LT) [Huang et al., 2021]. SP was also evaluated in the work. However, there was a bug in the code that likely strongly affected the results for SP [Goodfriend, 2024].

RAISocketAI In 2023 RAISocketAI was the first DRL agent which won the IEEE MicroRTS Competition. RAISocketAI includes a re-implementation of MicroRTS-Py, which is used in our code. Among other things, data generation via replays is an important basis for BC [Huft, 2025], and a bug in the old implementation was fixed that marked non-owned resources as owned resources when the agent is player 2. According to Goodfriend, this likely caused SP in the previous MicroRTS-Py implementation to yield no improvements, The RAISocketAI implementation of FSP achieved an average win rate of 91% against the same MicroRTS built-in bots as in MicroRTS-Py, which is significantly higher than 84% in the previous implementation. However, FSP achieved only a 20% win rate against CoacAI, whereas the best previous implementations were almost always able to defeat CoacAI. With fine-tuning on CoacAI and Mayari, the average win rate against the built-in bots was increased to 98% [Goodfriend, 2024]. LT was not implemented or evaluated in the work.

BeClone

BeClone demonstrated the effectiveness of BC with DRL fine-tuning in MicroRTS-Py using a two-phase training strategy for RTS agents: first, an initial policy is learned from experts via BC, and subsequently an RL fine-tuning is performed using *Advantage Actor-Critic* (A2C) [Martin and Jhala, 2021]. **AlphaStar** [Vinyals et al., 2019]

AlphaStar demonstrates that LT can be decisively contribute to robustness in complex RTS domains. The agent in AlphaStar was pre-trained with *imitation learning* from human replays.In this thesis, we replace this with the BC phases and the PPO phase. In both AlphaStar and this thesis the agent was further trained in a league of different agent types (Main Agent, Main Exploiters, League Exploiters, and historical snapshots (*historical agents*)) to reduce overfitting and forgetting and to promote diversity. Although MicroRTS and StarCraft II substantially differ in complexity and representation, AlphaStar is an established reference design for stabilizing and generalizing SP through league-based matchmaking mechanisms and the inclusion of specialized exploiters [Vinyals et al., 2019]. As in AlphaStar, in this thesis Main Exploiters are trained by training the base agent against all historical versions of the Main Agenten in the league, in order to identify and exploit weaknesses. At the same time, League Exploiters train against all historical agents. The Main Agent trains with *Prioritized fictitious self-play* PFSP against historical versions of itself, Main Exploiters, and League Exploiters. The prioritization of the historical opponents is computed based on the Main Agent's win rate against the respective historical agents. In this thesis, the prioritization formula was adjusted to better fit the MicroRTS environment. An exploiter is added to the league as a historical agent only when it shows a high win rate against all opponents or when a predefined step limit is reached [Vinyals et al., 2019]. AlphaStar performs policy updates using an *actor–critic method* with an off-policy correction mechanism called *V-trace* [Espeholt et al., 2018, Vinyals et al., 2019]. This approach is better suited for LT than PPO due to its robustness to off-policy data and its typically better scalability in large distributed setups. However, the implementation of PPO in MicroRTS-Py is significantly simpler and faster to realize, which is why PPO is a practical choice for implementing LT [Vinyals et al., 2019]. In addition, AlphaStar uses TD(λ) [Sutton and Barto, 1998] for value estimation value-function estimation.

3. Methods

3.1. MicroRTS-environment

MicroRTS is a two-player zero-sum game that is played on a two-dimensional grid (the map). Different units can be located on each cell of this map, and players can execute actions to move/train/attack units and to collect resources. The different units are:

- **Base:** 5 The Base can store resources and train Workers. It is very important, since it is the only way to store resources. (Cost (in resources): 10, hit points (hp): 10)
- **Resources:** 25 There is only one type of resource. It can be stored by Workers in the Base. (Number of resources in a cell: depends on the map in our case 25)
- **Worker:** Workers can collect resources and build Barracks. (Cost: 1, hp: 1, attack strength (at): 1)
- **Barracks:** Barracks can train Light, Heavy, and Ranged units. (Cost: 5, hp: 4)
- **Light:** Light units can move quickly, are trained quickly, and cost little. (Cost: 2, hp: 4, at: 2)
- **Heavy:** Heavy units can move slowly are trained slowly and cost a lot, but have high at and hp. (Cost: 3, hp: 8, at: 4)

- **Ranged:** Ranged units have an attack range of 3 cells. (Cost: 2, hp: 1, at: 1)

Worker, Light, Heavy, and Ranged units can move one cell in one of four directions. Units of player 1 are displayed with a blue outline, whereas units of player 2 are outlined in red. Most games are decided within 2000 steps. Therefore, the maximum number of steps is set to 2000. Movement is visualized with a gray line, attacking with a red line, and gathering with a green line. The experiments in this thesis were conducted using the RAISocketAI re-implementation of MicroRTS and the map `basesWorkers16x16A`, which is shown in Figure 1. This symmetric map starts each player with one Base, five resources, two Workers, and 50 resources near the Base. [**Goodfriend, 2024**]

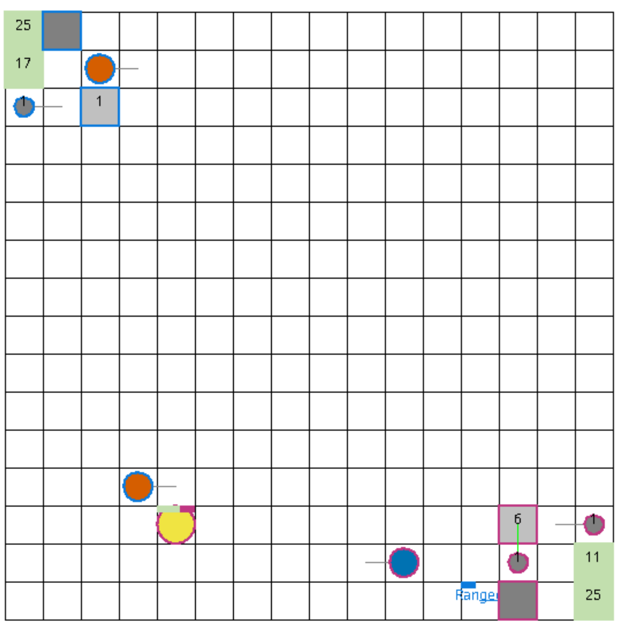

Figure 1. Screenshot `basesWorkers16x16A` map [Huft, 2025].

The action space used in this thesis is implemented using the combination of GridNet and invalid action masking [Huang et al., 2021]. GridNet produces a unit action (an action for one specific cell) for each cell of the map, regardless of whether a unit is located on the cell and whether the action is valid [Han et al., 2019]. Building on this, invalid action masking is used to mask invalid actions, thereby also masking actions empty cells [Huang et al., 2021].

The *unit action space* is defined as an 8-dimensional vector per cell:

$$(\text{pos}, \text{type}, d_{\text{move}}, d_{\text{harvest}}, d_{\text{return}}, d_{\text{produce}}, \text{pType}, \text{relPos})$$

The components of the unit-action vector are defined as follows:

- **pos:** Linearized index of the cell in the grid (i.e., the grid represented in a one-dimensional space; e.g., for a 16×16 grid: range: [0, 255]).
- **type:** Action type (range: [0, 5], 0: Nop, 1: Move, 2 Harvest, 3: Return, 4: Produce, 5: Attack).
- $d_{\textbf{move}}, d_{\textbf{harvest}}, d_{\textbf{return}}, d_{\textbf{produce}}$**:** Directions (range: [0, 3], 0: North, 1: East, 2: South, 3: West) for the respective actions.
- **pType:** Type of the unit to be produced. (range: [0, 6], 0: Resource, 1: Base, 2: Barracks, 3: Worker, 4: Light, 5: Heavy, 6: Ranged).
- **relPos:** Relative attack position. (range: [0, 48]).

For our agent, the unit action space (without the position dimension) is linearized and represented using one-hot encoding. The position is implicitly encoded by the linear grid indexing of the GridNet output [Huang et al., 2021]. Given a map of size $h \times w$, the action space has shape $h \times w \times 78$. For our 16×16 map, the action space per environment in the GridNet representation would therefore contain $16 \times 16 \times 78 = 19968$ actions. However, many of them can be invalid and are masked by invalid action masking [Huang et al., 2021]. Afterward the position is incorporated before masking the actions, otherwise the masking would remove the structure of the Action-Space, which would remove the implicit encoding for the position. Depending on the action type, only parts of the action vector are relevant (e.g., if the action type is `Move`, then only the components `pos`, `type`, and $d_{\texttt{move}}$ are relevant (the other components are ignored)).

3.2. Training Pipeline

The training pipeline extended in this thesis consists of four phases: BC, BC-FT, PPO, and LT (or FSP/SP). The first three phases were already implemented in the previous work [Huft, 2025] and are only briefly described here. In BC/BC-FT, demonstrations from experts (here, the built-in bots CoacAI and Mayari) are used to train an initial policy to learn basic game mechanics/strategies [Huft, 2025]. The BC phases reduces the initial, often inefficient exploration in DRL methodes such as PPO, which can be very time-consuming when trained from scratch [Torabi et al., 2018, Martin and Jhala, 2021]. The third phase improves the generalization and performance of the agent through additional exploration beyond expert demonstrations [Martin and Jhala, 2021]. PPO is an on-policy policy optimization method that constrains the size of policy updates in order to achieve stable training

[Schulman et al., 2017]. Using SP/FSP/LT, we aim to further generalize the base agent by training PPO not only against the built-in bots of MicroRTS (against which the base agent already achieves a high win rate), but also against opponents of equal strength or stronger. However, many opponents at this level of performance are not suitable for training, since they are very slow and thus slow down the collection of rollouts for PPO. In addition, the agent's generalization capability improves only to a limited extent when training against only a few new opponents, if the training environment does not provide sufficient opponent diversity. SP/FSP/LT address both problems by providing numerous opponents that continue to evolve over the course of training and achieve a high win rate against historical versions of the current agent.

3.3. self-play (SP)

In SP, the agent trains against its own current version, which leads to a continuous learning process with a co-evolving opponent [Silver et al., 2018]. However, training is often unstable because of cyclical learning. For example, if policy B performs well against A, C performs well against B, and A performs well against C, the agent may be trapped in a cycle in which it repeatedly relearns only the policy that performs well against itself, but not against other opponents [Vinyals et al., 2019]. To implement SP in the MicroRTS-Py reimplementation Huang et al. implemented the environment to output the observations (obs) and rewards for both players, which allows the agent to train against itself by simply using the same policy for both players [Huang et al., 2021]. In RAISocketAI, that approach was improved and fixed [Goodfriend, 2024]. In this Thesis, the same approach with some adustments is used to implement SP, which is also used as a basis for FSP and LT. A big problem in the SP training setup is that the agent has to learn to play as player 2 as well as player 1, which complicates the training and can lead to a slow learning process. The biggest difference between player 1 and player 2 is that player 1 starts at the top left of the map, whereas player 2 starts at the bottom right. This means that the agent has to learn to start play in two different areas of the map, which can be difficult and time-consuming. The problem is further exacerbated by the fact the training of this thesis does not start from scratch, but from a base agent that was only trained as player 1, which means that the agent would have to unlearn the bias towards starting at the top left of the map and learn to start at the bottom right as well. To address this problem, the observations for player 2 are adjusted by mirroring them along the diagonal of the map, meaning that in the observations for the agent, player 2 starts at the top left of the map as well. This makes the observations for player 1 and player 2 more similar and thefore making SP with an pre-trained base agent possible, even though the base agent was only trained as player 1. To mirror the observations the GridNet representation where flipped for every player two environment. Then the unit actions in the observations where flipped individually.

Actions are adjusted accordingly, so that the environment still executes the actions for player 2. Masks have to be adjusted as well, since they are based on the environments observations. The same mirroring is applied for the invalid action masks and the observations, while actions, including the position component of the action space was adjusted after the masking.

3.4. fictitious self-play (FSP)

To address the instabilities of pure SP, in FSP the agent is trained not exclusively against the current policy, but against a uniformly sampled pool of the agent's historical policies. This results in the learning agent learning a good response to the averaged opponent strategy, which reduces cyclical learning. Under standard assumptions, FSP can converge towards an approximate Nash equilibrium in two-player zero-sum games (with respect to the average strategies). Heinrich et al. also demonstrate this empirically in poker experiments [Heinrich et al., 2015]. The implementation is similar to SP, but instead of Player 2 always using the current policy, Player 2 is sampled uniformly from a pool of historical agents (snapshots of the Main Agent at different training stages). If we implement it in this way, a problem arises: player 1 has priority in conflict resolution (e.g. when both players want to move to the same cell). This gives player 2 a disadvantage, although it has only a minor effect on the results. The actual problem is that the agent can exploit this prioritization in FSP if it learns that optimizing the training objective can be achieved primarily by improving player 1, while deliberately performing poorly as player 2. After sufficient training, the agent will learn a strategy that is based on player 1 being prioritized. For example, the agent could learn to block an opponent in front of the Base by issuing a simultaneous move to the same target cell. In FSP, player 2 would then be blocked, while player 1 would not, allowing the agent to achieve a win rate close to 100% against itself. Training then stagnates, since the agent already has a hight winrate against itself without develping a robust strategy against other opponents. In LT, this is not quite as problematic because there are also exploiters that can be trained against, which remain adversarial to the Main Agent. However, a large portion of the training becomes ineffective and provides poor learning signals if the agent exploits the player 1 priority. The Problem is particularly relevant for LT, which combines SP and FSP, because in only historical Agents are available for training in FSP, making the feedback for player 2 very sparse, while in LT

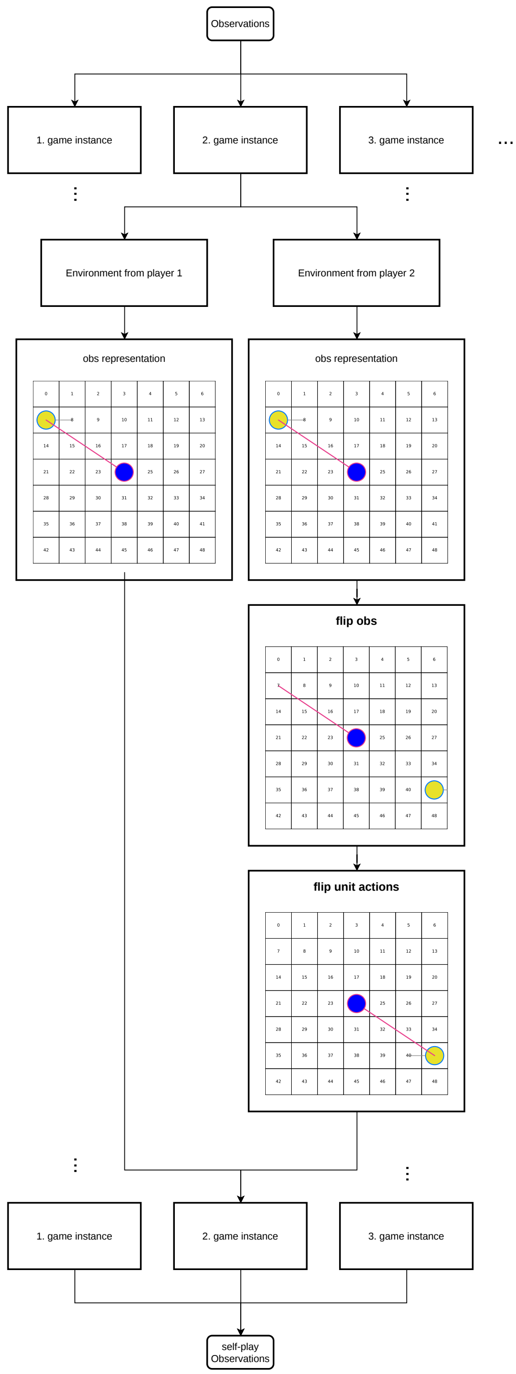

Figure 2. Observation adjustment for player 2 in self-play.

the Main Agent can directly train against the current Main Agent. This yieldes more immediate feedback on the Main Agents performance as an opponent (player 2). As a result the agent no longer has to plan as far ahead in order to exploit this priority and minimize its own win rate as player 2.

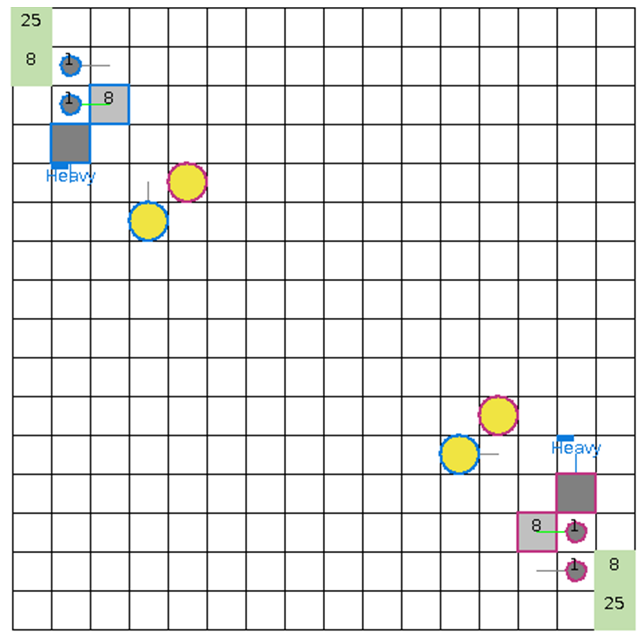

Figure 3. Screenshot showing an action conflict in the game.

In the re-implementation of MicroRTS-Py, there is a mode that resolves conflicts in a randomized manner. However, this does not help in this case, because even before the conflict arises, invalid action masking masks out the action of player 2 since it is invalid. To solve this problem, the learning agents in the parallel games are alternated between playing as player 1 and player 2. This does not prevent the prioritization of player 1. However, it can be shown empirically that the prioritization of player 1 has no major impact on the results. This prevents the exploitation of the prioritization by the learning agent, since it can now also be player 2 and therefore a strategy that is only effective for player 1 is no longer beneficial.

3.5. prioritized fictitious self-play (PFSP)

LT uses an extended variant of FSP called PFSP [Vinyals et al., 2019]. PFSP extends FSP by selecting opponents against which the agent can train efficiently, rather than selecting opponents uniformly. The selection probability is based on the win rate of the learning agent against the potential opponent [Vinyals et al., 2019]. The exact prioritization function can differ for different agent types. In AlphaStar, for the Main Agent, opponents of medium difficulty (win rate close to 50%) are prioritized. While exploiters prioritize the strongest opponent (against which the win rate is closest to 0%). Adapted prioritization functions tailored to our training setup are used in this thesis, which deviate from those used in AlphaStar. An important difference is that training against an opponent against which the base agent has a win rate close to 0% yields little informative gradient or learning signal. To avoid such opponents, the following prioritization function was used for the Main Exploiters:

As can be seen in the figure, we prioritize opponents with a low win rate not close to 0%. Furthermore, opponents with a win rate of 0% are prioritized over those with a win rate of 100%, since there is substantial learning potential for an opponent with a win rate of 0%, whereas a win rate of 100% usually means that the agent already defeats the opponent reliably. What is not shown in the figure is that all agents have a non-zero probability of being selected. This ensures that no opponent is completely ignored because it lost all initial games (yielding a 100% win rate for the learning agent), since it is likely that those initial results were outliers.

3.6. league training (LT)

LT also extends FSP by not only training on historical agents, but instead by training on an entire league of agents, including Historical agents, main exploiters (that are only trained against historical Main Agents) and league exploiters (that are only trained against all historical agents) [Vinyals et al., 2019].

LT utilizes FSP with PFSP, to find the best historical opponent to train against.

3.7. League Training (LT)

League Training (LT) is a population-based extension of self-play designed to reduce the non-stationarity and cyclical behavior of pure SP by training against a diverse set of opponents that represent different stages and styles of play [Vinyals et al., 2019]. Instead of facing only the current policy (SP) or a uniformly sampled set of past policies (FSP), the learning agent is matched against opponents drawn from a *league* using *Prioritized Fictitious Self-Play* (PFSP) (Section **??**). This yields a training curriculum in which the Main Agent repeatedly encounters (i) historical versions of itself, and (ii) specialized opponents that actively search for and exploit weaknesses of the current policy.

League composition and roles. Following the general design of AlphaStar [Vinyals et al., 2019], the league in this thesis consists of:

- **Main Agent (MA):** the primary policy that is optimized and evaluated.
- **Historical Agents (HA):** frozen snapshots of earlier Main Agent parameters. They stabilize training by providing a stationary distribution of past strategies and mitigate catastrophic forgetting.
- **Main Exploiters (ME):** agents trained to beat the Main Agent by emphasizing opponents (historical snapshots of MA) for which the exploiter currently performs poorly. Conceptually, MEs encourage the discovery of blind spots in the current MA policy.
- **League Exploiters (LE):** agents trained against the full league distribution (historical agents and exploiters). This promotes strategic diversity and prevents the league from collapsing to a narrow set of responses.

All agents share the same observation/action representation (GridNet with invalid action masking) and are trained with PPO; the difference between roles is solely the opponent sampling distribution.

Matchmaking with PFSP. For each PPO rollout batch, an opponent policy is sampled from a candidate set (historical agents and exploiters) according to a PFSP distribution (Section **??**). The sampling probabilities are computed from estimated win rates between the learning agent and each candidate opponent (updated periodically from evaluation matches). In contrast to uniform FSP, PFSP allows us to focus training on opponents that provide informative gradients while still maintaining coverage of the entire league via a small non-zero minimum selection probability. This avoids permanently ignoring opponents that happened to lose early matches due to variance, and it prevents training from stalling on opponents that yield little learning signal.

Training loop and snapshot management. LT proceeds iteratively in cycles consisting of (i) matchmaking, (ii) PPO updates, (iii) win-rate estimation, and (iv) league updates:

1. **Select opponent:** Sample an opponent from the league using a role-specific PFSP distribution.
2. **Collect rollouts:** Run self-contained matches between the learning agent and the sampled opponent for a fixed number of environment steps. During rollouts, the opponent policy is frozen (inference only).
3. **Update policy:** Perform PPO updates using trajectories collected against the sampled opponent.
4. **Update statistics:** Periodically evaluate against a subset of league members to update win-rate estimates used by PFSP.
5. **Add snapshots:** At predefined intervals (or when performance criteria are met), store a frozen copy of the current agent as a new historical agent.

In practice, snapshotting implements a memory mechanism: even if the Main Agent changes rapidly due to PPO updates, the historical pool preserves earlier strategies and thereby stabilizes the opponent distribution over time.

Why exploiters are needed. A league containing only historical Main Agent snapshots can still drift towards strategies that are strong only against the averaged historical distribution. Exploiters counteract this by explicitly searching for vulnerabilities: if the Main Agent exhibits systematic weaknesses (e.g., against specific rush timings or unit compositions), exploiters can amplify those weaknesses into consistent counter-strategies. Training the Main Agent against these exploiters encourages robust policies that generalize better to unseen opponents than policies trained with SP/FSP alone [Vinyals et al., 2019].

Practical considerations in MicroRTS. In MicroRTS, generating PPO rollouts against very strong search-based opponents can be prohibitively slow. LT provides a scalable alternative because league opponents are neural policies (historical snapshots and exploiters) with fast inference, enabling efficient data collection while maintaining opponent diversity. Moreover, the explicit control over opponent sampling (PFSP) is crucial in MicroRTS because certain opponent matchups can lead to long stalemates near the episode length limit, reducing sample efficiency; therefore, the prioritization functions in this thesis were adapted to emphasize opponents that yield informative learning signal while still preserving diversity through non-zero exploration over the league.

4. Experimente

Table 2. Training parameters for PPO training.

Parameter	Value
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

- **Generalization to random opponents:**
- **Map and observability constraints:**

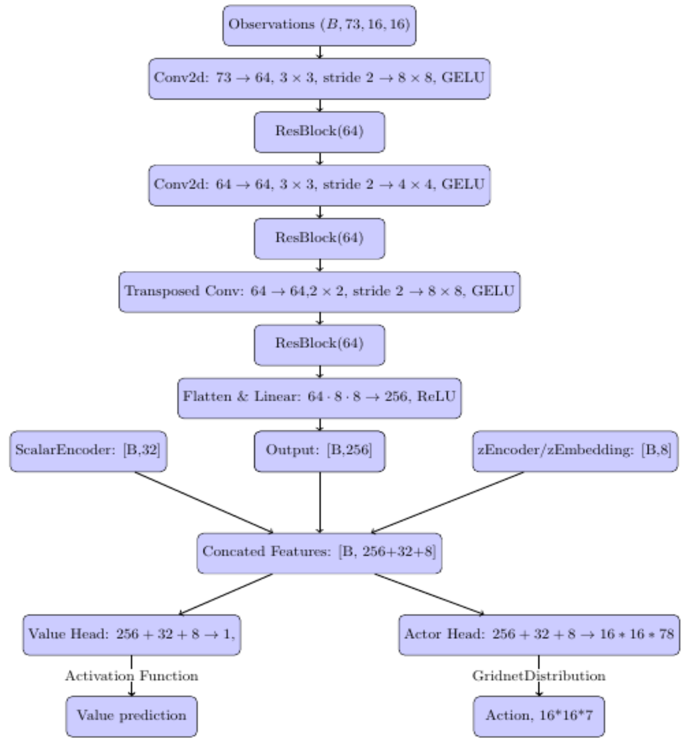

Figure 4. The Network Architecture

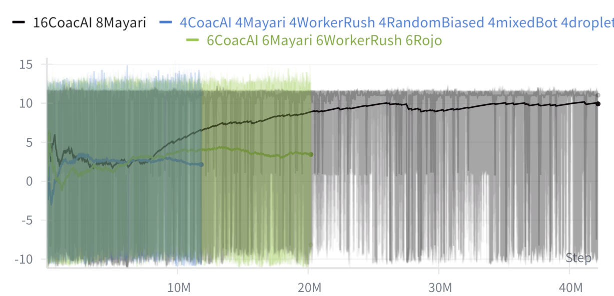

Figure 5. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the BC-FT agent.

- **Replay and training bottlenecks:**
- **Exploitability:**

5. Conclusion

References

[Dhariwal et al., 2017] Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., Wu, Y., and Zhokhov, P. (2017). Openai baselines. https://github.com/openai/baselines. 2

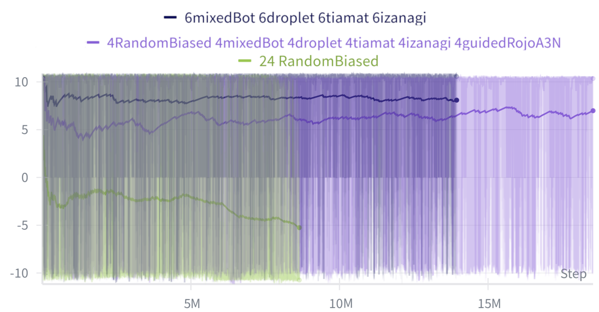

Figure 6. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the final agent.

[Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 2, 3

[Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the ieee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 4, 5

[Han et al., 2019] Han, L., Sun, P., Du, Y., Xiong, J., Wang, Q., Sun, X., Liu, H., and Zhang, T. (2019). Grid-wise control for multi-agent reinforcement learning in video game AI. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2576–2585. PMLR. 2, 4

[Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 1, 5

[Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 1, 2, 3, 4, 5

[Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 2, 3, 4

[Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 3, 4

[Ontañón, 2013] Ontañón, S. (2013). The combinatorial multi-armed bandit problem and its application to real-time strategy games. In *Proceedings of the Ninth Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 9, pages 58–64. AAAI. 2

[Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2, 5

[Silver et al., 2018] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362:1140–1144. 5

[Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. The MIT Press. 3

[Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2, 4

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 1, 3, 5, 6, 7, 8

Erklärung

GPT!! Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den XXXXXXXXXXX

Name!!